

Concrete Architecture - Gemini CLI

Group Name: The Rat Pack

Aatman Gupta 22ag32@queensu.ca

Caleb Gao 21cg49@queensu.ca

Cory Boys (presenter) 21alb25@queensu.ca

Finnegan MacAdams (presenter) 23bmd@queensu.ca

Jude Campanelli-Henderson (leader) 22nbx2@queensu.ca

Rémy Brown-Vandervecht 22WLHR@queensu.ca

CISC 322

Professor Bram Adams

TA: Ayesha Babar

Abstract

This report presents a detailed analysis of the architecture of Gemini CLI. The report begins with comparing Gemini CLI's concrete architecture to our previously derived conceptual architecture from Assignment 1 (A1). Our concrete architecture was derived using the SciTools Understand software to examine the source code. It allowed us to identify the important systems and subsystems and their connections. Our findings showed that most components of our conceptual architecture were correct, but some dependencies between components were omitted. We then analyzed the inner architecture of the CLI package, which parses user input and communicates with the Gemini API via supporting tools, configuration, and security modules. The report then examines two use cases, each illustrated with sequence diagrams. The first use case describes how a developer can use the Gemini CLI to test and debug a codebase. It demonstrates how the system executes tests through integrated tools and iteratively suggests code fixes. The second use case describes a time when a user asks the Gemini CLI to create a file containing a program that meets the requirements or specifications listed on a URL. This demonstrates the system's use of the Web Fetching Tool, File System tools, and security components.

Introduction

The goal of this report is to document the concrete architecture of Gemini CLI. We have presented our analysis based on an examination of the source code using the SciTools Understand analysis tool. In the software, we explored the provided repository and created dependency diagrams to obtain our system and subsystem overviews. The diagrams created and provided identify the major subsystems and their interactions. We used the Understand tool's architecture design feature to see the real dependencies of the Gemini CLI.

We also study the inner architecture of the CLI package. The CLI package collects user input and then interacts with the Gemini API package to provide a response. The CLI package begins with initializing a main program that handles error management. It then runs a test setup function for Vitest, which is then configured to define how tests should be discovered, executed and reported for the program.

We also used programs outside Understand to create a high-level representation of the Gemini CLI's concrete architecture and compared it to our previously proposed conceptual architecture. In A1, we chose the client-server model, which we found consistent with our Understand analysis. The CLI package acts as the client to request services from the central server, and manages the user interactions. The core package serves as the backend that handles tool execution. This client-server architecture benefits from modularity. However, we found our second identified style in A1 to be inconsistent through feedback and Understand analysis.

System Overview

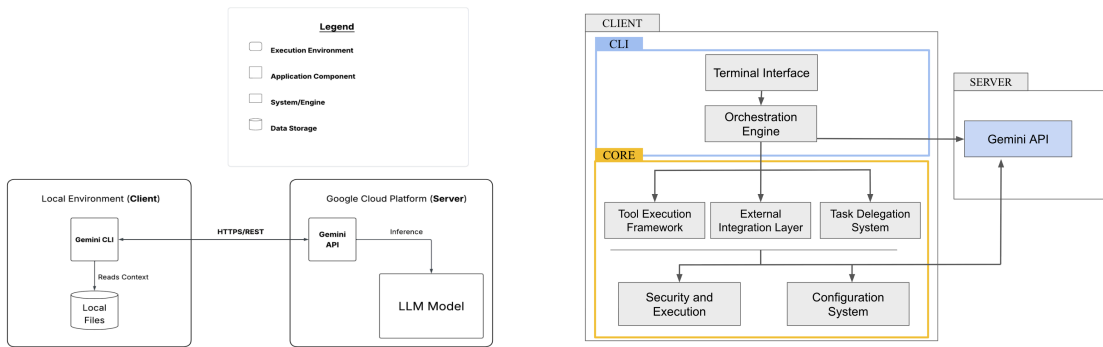


Figure 1 and 1.1. High level conceptual architecture, box and arrow diagrams

This is a proposed high-level conceptual architecture that provides more specific components than we used in our original A1 conceptual architecture overview and follows a similar structure. The initial architectural styles proposed are the client-server model and the feedback control system. In the client-server model, the system is divided into two independent entities connected via a network. The client is the local Gemini CLI running on the user's machine, and the server is Google's Gemini API infrastructure. In interactions between these entities, the client would send queries to the server, which would perform the heavier computations and then return the responses to the client.

The feedback control system originally proposed would be better explained using the layered architecture style. Within the CLI and Core packages, the system employs a layered style to facilitate simple communication between processes and to separate different types of functions. Within the CLI, there is a distinct separation among presentation, state management, and logic; in the Core, the orchestration, content generation, and extension layers are separated.

Terminal Interface consists of the CLI component, managing all user input, display, and formatting the response that is sent back from the Core Component.

The Orchestration Engine is responsible for all backend logic of the system, orchestrating tool use, tasks, and prompts sent to the API.

Tool Execution Framework manages tool use, and stores a repository of available tools and their logic.

External Integration Layer enables the integration of external tools and resources. It communicates with the tool execution framework and external servers, as well as the orchestration engine.

Task Delegation System determines which system should execute each task, and sends results to the next system to perform the next required task.

Security and Execution is responsible for the safety of the responses provided by the Core component. It ensures that any proposed response follows policy and is safe to execute.

Configuration System provides the settings, context files, and policy, which can be customizable by the user, to the components.

Derivation Process

We used SciTools' Understand architecture design feature, which allowed us to see the real dependencies for the components highlighted in the box and arrow diagram in figure 1 above. These components were derived from the global A1 feedback provided to the class, and the documentation found on Github, we have made our analysis of the high level concrete architecture to include more specific components, and have updated the names accordingly. Below is a screenshot from Understand, showing these components and their dependencies. We added

gemini-v2/packages/core/src/config for the **integration system component**

gemini-v2/packages/cli for the **orchestration component**

gemini-v2/packages/core/src/safety for **security and execution**

gemini-v2/packages/a2a-server for **servers**

gemini-v2/packages/core/src/scheduler for the **task delegation system**

gemini-v2/packages/cli/ui for the **terminal interface**

gemini-v2/packages/core/src/tools for the **tools execution framework**.

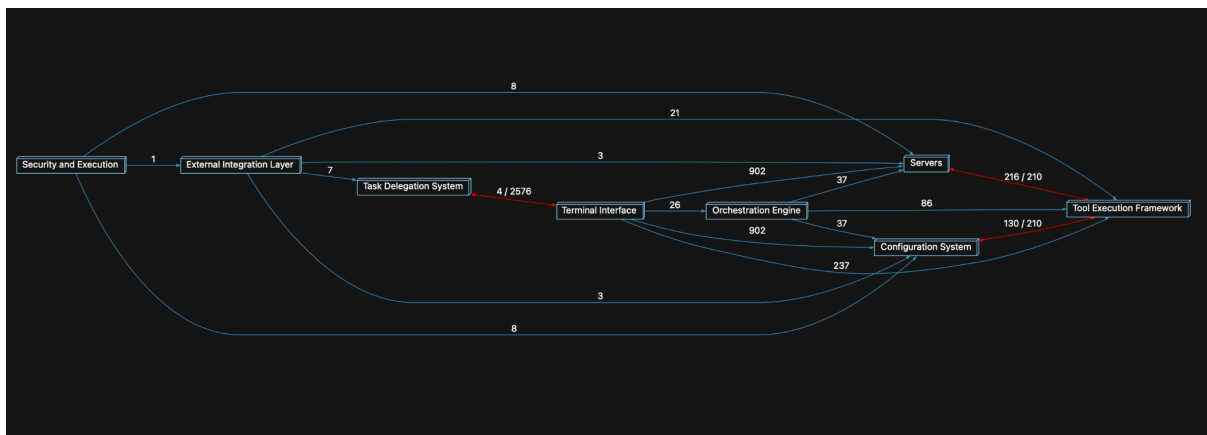


Figure 2. Architecture screenshot from Understand

Reflexion Analysis

The concrete architecture derived using Understand and Gemini CLI's source code is similar to the conceptual architecture proposed earlier. The most differences were found in some dependencies between components in the service layer that were not included, and some dependencies that were bidirectional that we had originally marked as unidirectional. Pictured is the resulting concrete architecture, reformatted to facilitate comparison between the conceptual architecture. Red arrows indicate dependencies where this is the case (a dependency was missed or mistakenly unidirectional), and grey arrows represent dependencies that were included.

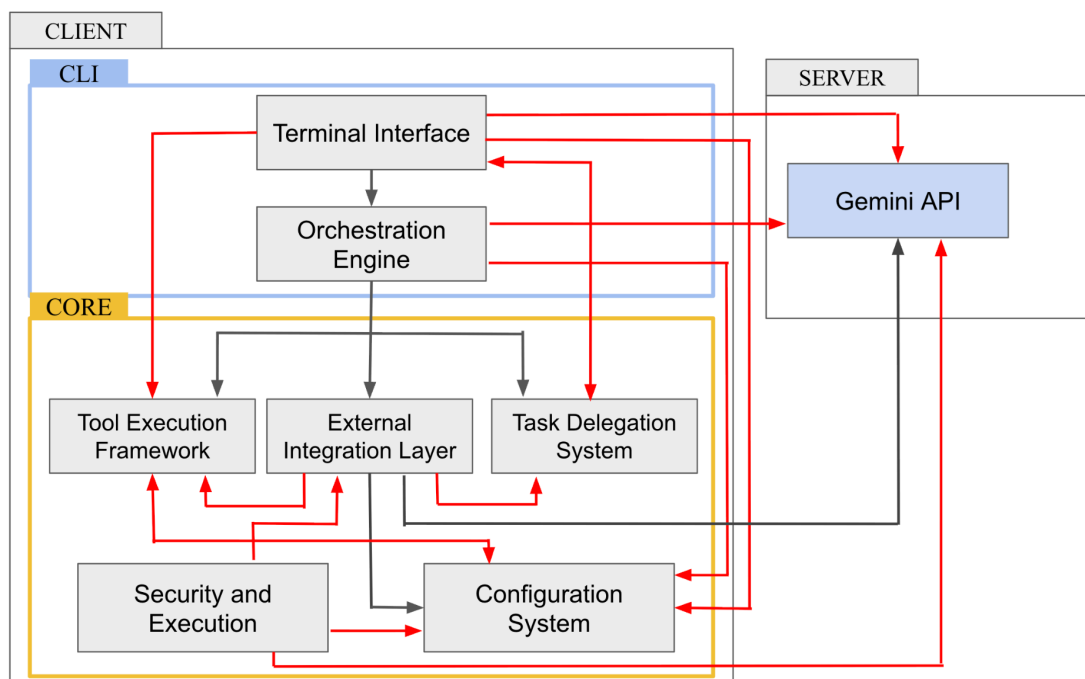


Figure 3. High level Concrete architecture, box and arrow diagram

Inner Architecture of a Subsystem

For our Inner Architecture analysis, we will break down the CLI package. The CLI Package parses the user's input and interfaces with Gemini as described by the diagrams below.

The CLI package starts with the index Node.js file, which is the CLI entrypoint that initializes the main program and handles error management & cleanup. It imports the main gemini source file, as well as the FatalError, writeToStderr and runExitCleanup packages from the core's utilities and attaches these processes to the main process, which listens for errors and logs them if any occur. There's also a built-in bug fix for a known race condition that occurs on 32-bit systems running windows.

Next, the CLI package runs `test-setup.ts`, which is a test setup function for Vitest. Vitest is a JavaScript testing program which helps developers build and enforce tests for their code, particularly for React components, which is a UI building framework for JavaScript.

Following this is `vitest.config.ts`, which is a configuration file for the aforementioned Vitest framework. It defines how tests should be discovered, executed and reported for the program. The script uses `defineConfig` from Vitest's API to allow packages to expose test-specific functions such as *describe*, *it*, and *expect*. The config file also sets up reporting and CI (continuous integration) through the JUnit reporter, which is a commonly used reporting system for test results. Code coverage is also implemented using the V8 JavaScript engine which collects execution data during test runs.

Finally, we have the `examples` directory containing a single script called `scrollable-list-demo.tsx`, which implements a terminal-based demo application built with React. The purpose is to demonstrate and test and custom UI components in the `ScrollableList` showing how it behaves with datasets and elements added by the program.

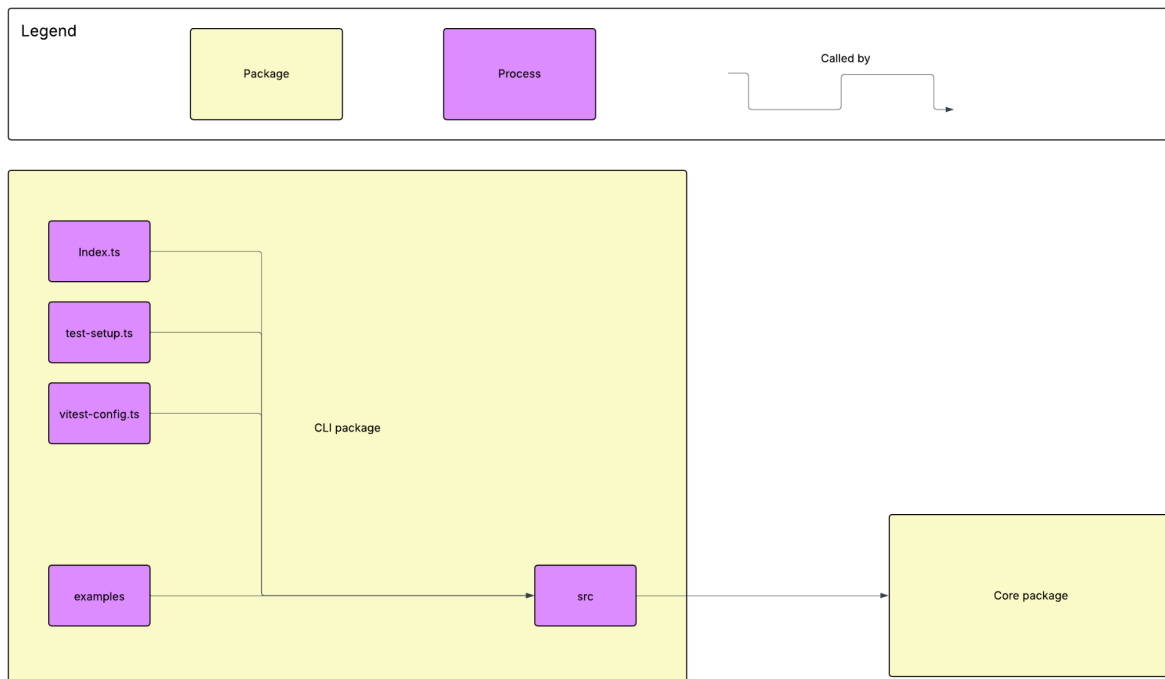


Figure 4. CLI package communication, box and arrow diagram

Use Cases Legend

Arrow Types:

- Synchronous method call (caller waits for reply)
- Asynchronous method call (caller does not wait for reply)
- - - - -> Return message/callback

Guard [condition]: Decision point

Lifeline Naming:

CliCommandHandler: CLI entrypoint (handles input/display)

AuthManager: Token validation and security

ConfigService: Loads GEMINI.md, settings, context

OrchestrationEngine: Workflow coordinator (your “brain”)

SessionHistory: Conversation state/chat history

ToolRegistry: Executes tools like “run_tests”, “fix_code.”

GeminiApiClient: Gemini model API calls

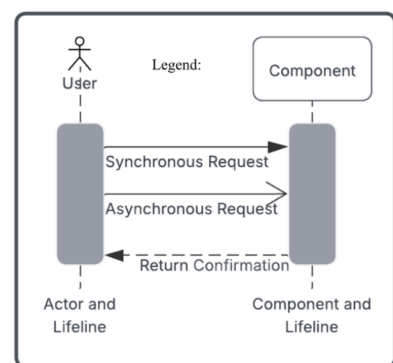


Figure 5. Sequence diagram legend

Use Cases 1: “Test and Debug Codebase”

The following diagram outlines how a developer interacts with an AI to ensure their code runs smoothly, through a cycle of testing, analyzing failures, suggesting fixes, and refining the code until it passes all tests.

It all begins when the developer sends a request through a tool called CliCommandHandler. The very first action is the AuthManager verifying the user’s token for security; next, the ConfigService loads the necessary session settings from the markdown document GEMINI.md, which contains all relevant information, content, and guidelines for the Gemini AI model. Once everything is in place, the OrchestrationEngine takes over. It creates a structured prompt that includes important information such as the session ID, available tools, and the history of previous commands. This prompt is then forwarded to GeminiApiClient to initiate a conversation with the AI.

As the AI responds, it identifies that tests need to be executed. To proceed, the OrchestrationEngine requests confirmation from the user via the CliCommandHandler, ensuring the user agrees to run the specified tool. If the user confirms, the system calls ToolRegistry to run the tests with the provided arguments, and the results are returned and processed.

Compared to the A1 conceptual architecture’s Use Case 1 sequence, our concrete version refines the high-level components into specific subsystems while preserving the core flow: Terminal input → authentication → configuration → orchestration → Gemini API → tool execution → user confirmation → response. We expanded OrchestrationEngine to include detailed interactions and added SessionHistory as an explicit lifeline to show conversation-state management, which was abstracted in the conceptual diagram. The [while tests failing] loop concretizes the conceptual “iterative tool usage” pattern.

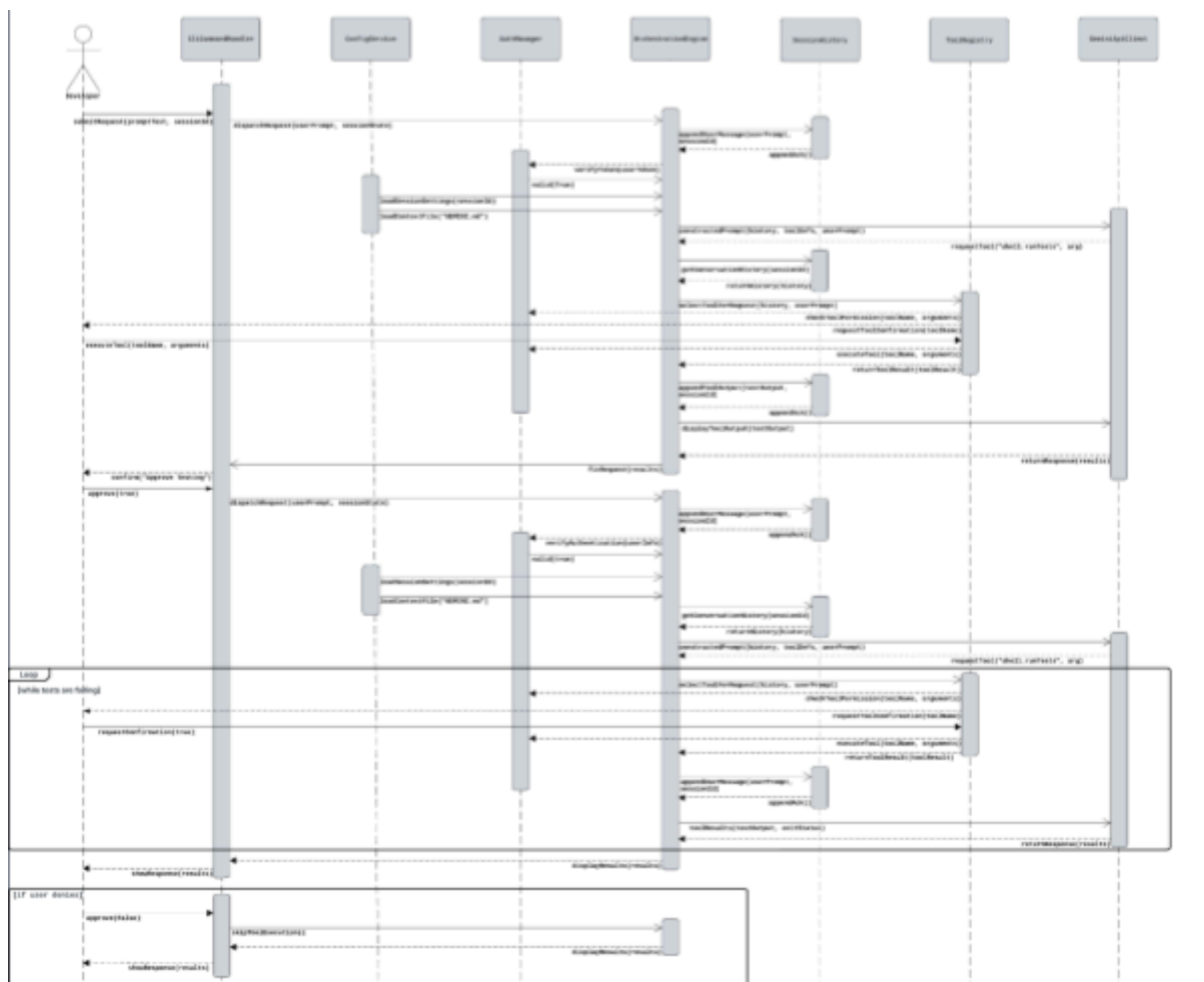


Figure 6. Sequence diagram for testing and debugging codebase

Use Case 2: “Create a program based on online requirements”

In Use Case 2, which involves creating files from web specifications, we decided to leave out the SessionHistory lifeline. This is because it operates on a straightforward single-pass pattern: the history is loaded just once at the start with the `getConversationHistory(sessionId)` function and then appended only once after the tool finishes its task using `appendToolOutput()`. Adding this lifeline would simply duplicate the interactions already shown without providing any new insight into the architecture. By keeping it out, we can maintain the focus on the main workflow of fetching data from the web and writing it to a file.

The diagram below illustrates a use case in which a user asks the AI agent to create a file containing a program that meets the requirements or specifications listed on a website or online blog, with the URL provided.

To begin the process, the developer (user) enters the request and URL in the terminal, which forwards them to the Orchestrations Engine in the Core package, verifying the user's authentication as it does so. The engine then formats the request into a structured prompt for the Gemini AI, adding previous context, available tools, and message history from Configuration settings and session context.

This expanded prompt, which provides the Gemini AI with a clearer understanding of the current development state for use, is then forwarded to the Gemini API, and a response is received.

Gemini's response requires the Web Fetching Tool, which is invoked by the tools component and fetches the requirements from the specified URL. In this use case, we assume, for simplicity, that the request succeeds without interruption and that the URL contains the requested information; if the URL were malformed, the tool would return an error state, prompting the user for additional instructions or an updated URL. Similarly, if the Gemini API received insufficient instructions from the scanned data, it would return a request for more precise specifications. The tool returns the scanned data to the Orchestration engine in the Core package, which then forwards it to the Gemini API again.

Using the scanned web information and the context provided by the Core package, the Gemini API proposes a solution that satisfies all requirements and requests the use of the File System tools to create the file. Since this involves modifying files, the Security Component requires user (developer) approval, which it requests from the terminal. The Core package reroutes this request to the CLI package, which then informs the user about what the AI intends to create or modify and prompts them to approve or deny. If the developer denies, the terminal reports that state to the Core package, which then terminates the process without creating a file and returns a summary message to the user.

If the user approves the proposed file, the Orchestration Engine invokes the File System tools to create the file and write the API's proposed solution to it. When that process is complete, the tools component returns the results to the engine, from which the Gemini API generates its final summary, including the details surrounding the completed file, its capabilities, how it fulfills the requirements outlined for it, and other relevant information and advice regarding the use of the newly created program. The engine then forwards this summary to the terminal, which displays it to the user.

Similar to Use Case 1, the concrete version refines our breakdown into subsystems that more closely reflect the flow detailed in our original conceptual architecture. We separated the Core and CLI packages into their more specific components, detailing their interactions in greater detail, particularly around authentication and security checks.

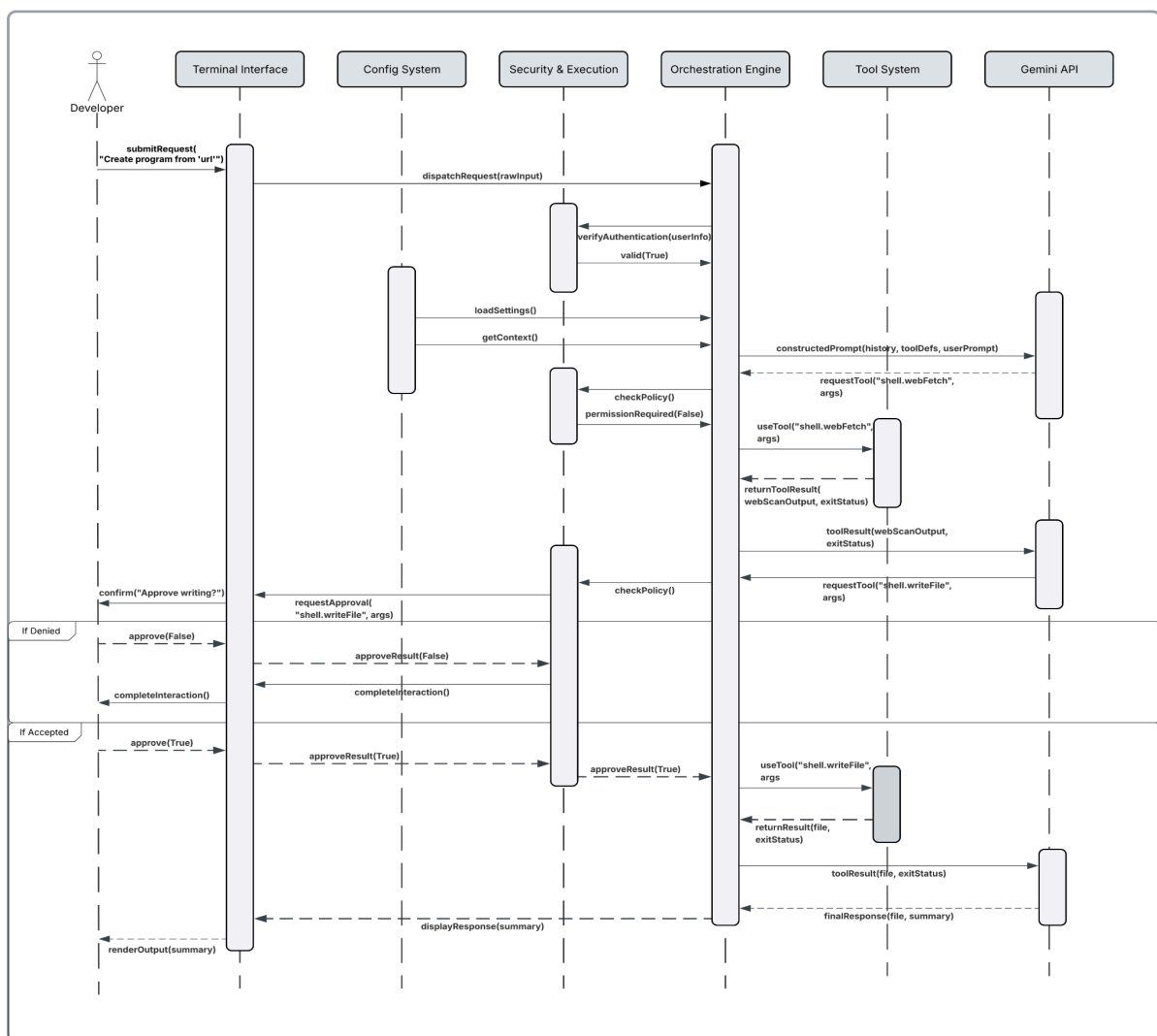


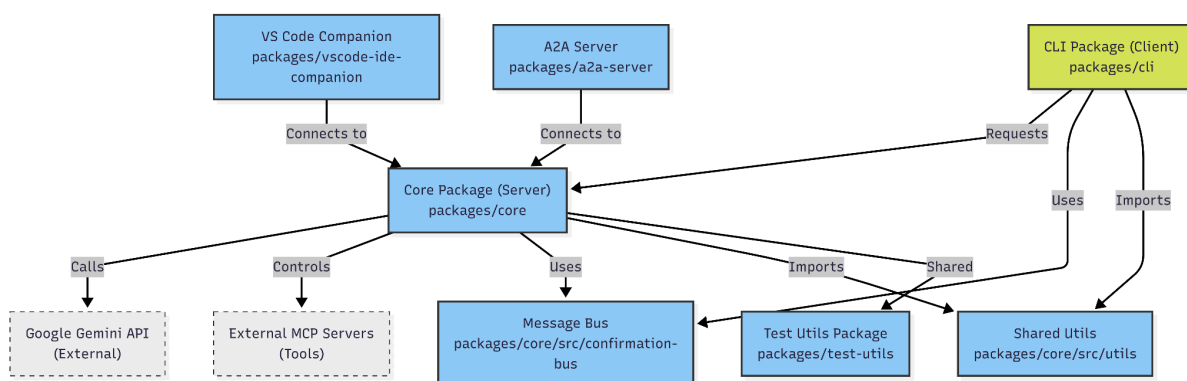
Figure 7. Sequence diagram for creating a program based on online requirements

Reflexion Analysis for High-Level Architecture

Our high-level reflection analysis shows that the client-server model was a very suitable choice for the architecture in our A1 report. The CLI package acts as a client that manages the user interface and user interactions. Meanwhile, the core package serves as the backend server that handles the LLM and tool execution. This separation is useful because it decouples the agent brain's complex logic from the CLI rendering system. It also makes the code much easier to maintain.

Divergence: The Tools Subsystem Structure

One major divergence we found is that the Tools subsystem isn't a peer component as we planned in A1. Instead of being a separate package, the tools are actually nested inside the core package at **packages/core/src/tools**. This is likely because the *CoreToolScheduler* needs to have direct control over tool state and security during the execution loop. If they were separate components, it would be much harder to manage the security and "Approval Pending" states required for a CLI agent. We even found [commit 21fba832](#) in the understand repository, where they renamed the backend to core to show they wanted to centralize the logic. This proves that they wanted everything under one roof for better control.



Reflection Analysis for 2nd-Level Architecture

The initial 2nd conceptual architecture we identified described Gemini CLI as a feedback-control system, where a central controller interpreted a user request and returned results as feedback. According to the concrete architecture of the git repository visualized by Understand, the CLI package operates as a client-server architecture, as correctly identified in the primary architecture analysis.

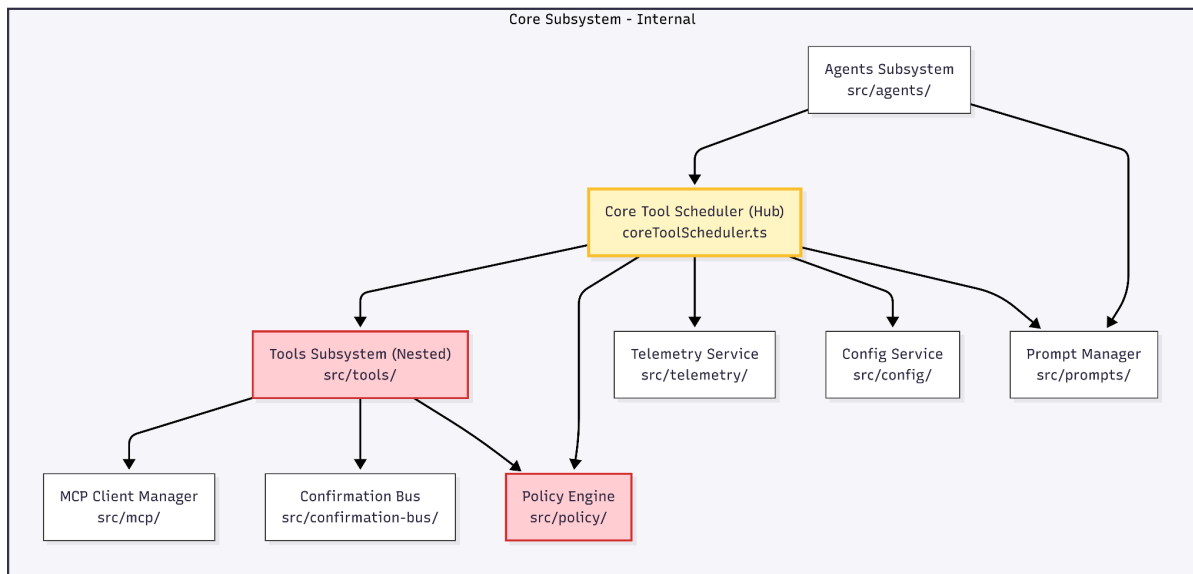
Divergence #1: Absence of an Environmental Feedback Loop

The conceptual architecture assumes that the system observes an environment and takes actions. In the Concrete architecture, however, the repository structure shows no dedicated observation of the environment.

Divergence #2: CLI as the Central Orchestrator

The conceptual model placed the core package as the controller, with the CLI acting only as a user interface. In the Concrete architecture, the dependency diagram indicates the opposite: the CLI is the central hub, and the other components connect through it.

The Conceptual architecture does, however, correctly identify a modular structure separating the CLI interface, the core logic, server components & testing tools shown in the concrete architecture repository.



AI Usage: Gustapo The Rat

For Assignment 2 (A2), we continued utilizing Perplexity AI (“Gustapo the Rat”) following the successful approach from A1. The shared Space, with its restricted search capabilities (limited to course materials and whitelisted Gemini CLI documentation), facilitated the collaborative refinement of our concrete architecture diagrams and use case explanations

Key Steps for A1:

1. **Uploading Materials:** We made sure to upload all relevant assets, including our course slides, the A1 report, Gemini CLI documentation, and the marking rubric, into the shared Space.
2. **Seeking Feedback:** We reached out to Gustapo for an evaluation of our drafts based on the rubric and asked for recommendations on how to improve.
3. **Independent Validation:** We took the feedback from Gustapo as potential ideas and checked them against our course materials and relevant repositories before making any changes.

Main Applications in A2:

Diagram Review: Gustapo helped us fine-tune our box-and-arrow and sequence diagrams, ensuring that everything, from lifeline mappings to method calls, was consistent with the overall architecture. It also pointed out an error we had for the boundaries of the Gemini API.

Refining Use Cases: The tool was beneficial in clarifying our explanation for Use Case 1, which deals with the testing loop. However, we decided to ignore one suggestion regarding loop positioning because it didn't align with our architectural goals.

Marking Scheme Simplification: We worked on simplifying the marking rubric into distinct checkpoints and carefully checked our submission against these to ensure we met all requirements.

Quality Control: Gustapo essentially performed the role of an "autograder," enabling us to compare its feedback with our primary sources. This process led to about a 15% improvement in our final text regarding explanations and comparisons, while we kept the diagrams and method names the same, based on our analysis of the repository and our use of the Understand tool. We always treated the AI-generated feedback as suggestions, rather than definitive answers.

This consistent methodology not only enhanced our understanding but also leveraged AI to increase efficiency, ensuring full traceability to the course materials.

Lessons Learned

Before writing our report, we learned a lot from the feedback provided to us on our A1 report. We learned that one of our identified architectural styles was incorrect. Based on the comments we learned how to translate our idea of the architecture, along with further research into the topic, to the correct understanding of the structure.

To create this report, our group had to first learn how to use the SciTools Understand application. Along with our better understanding of the architecture, we learned to construct and view the system overviews, and most included diagrams. We found our problem solving skills were important when using a new tool. Collaboration in person is also something we are continuing to learn is important to our success. While meeting in person we could ask each other questions about the software; and were overall more productive.

Conclusion

Our analysis of Gemini CLI's concrete architecture demonstrated how the system was actually structured. In comparison to our previously proposed A1, reflexion analysis showed that several aspects of the original conceptual architecture were correct but not all. While having most of the same components, we underestimated how interconnected the architecture

truly was. We did not predict a few of the dependencies between components in the service layer, and some dependencies we had previously believed to be unidirectional that were actually bidirectional.

The analyzed use cases further illustrate how Gemini CLI's architecture supports real-world development tasks. The first use case demonstrates how a developer can use Gemini CLI to test and debug a codebase. More specifically, both use cases also demonstrate how user requests flow through the CLI interface to the core orchestration system, how the Gemini API and external tool components are used to gather information or perform actions, and how security and authorization is used to protect its users and their data.

References

<https://vitest.dev/>

<https://react.dev/>